Reinforcement Learning

Class 4

Model – Free Learning:

Monte Carlo and Temporal Differences

Mostly based on D. Silver (UCL) lecture 5 from 2015
and sections 5.1-5.2/6.1-6.3/7.1/12.1-12.6 from Sutton & Barto's book

Context

- Last classes – we presented MDPs as the mathematical framework underlying RL problems, and next Dynamic Programming as a way to solve known MDPs
- This class – we will address methods to **estimate the value functions of unknown MDPs** (**model-free prediction**): Monte-Carlo and temporal-difference learning
- Next class: we will review methods to optimise the value functions of unknown MDPs (**model-free control**)
- We mean model-free –> unknown environment/ MDP

Monte Carlo methods for RL

- MC methods learn directly from **experience** – assumed to be divided into **episodes**
- All episodes terminate no matter what actions are selected.
- MC is **model-free**: no knowledge of MDP transitions / rewards is necessary
- MC learns from complete episodes: no bootstrapping
- MC uses the simplest possible idea: **value = mean return**
- So, we will take the mean of the values of the different episodes; value estimates are only updated in the end of each episode
- Caveat: can only apply MC to episodic MDPs

Monte Carlo – policy evaluation

- Goal: learn v_π from episodes of experience under a **given policy π**

  $S_1, A_1, R_2, \ldots, S_k \sim \pi$

- Recall that the return is the total discounted reward:

  $G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T$

  and that the value function is the expected return:

$$v_\pi(s) = \mathbb{E}_\pi \left[G_t \mid S_t = s \right]$$

- Monte-Carlo policy evaluation replaces expected values by using empirical mean return
- They will sample and average returns for states like the bandits sample and average rewards (class 2)

First-Visit Monte-Carlo Policy Evaluation

- To evaluate state *s*, the **first time-step *t* that state *s* is visited** in an episode: over a set of episodes,
  - increment counter $N(s) \leftarrow N(s) + 1$
  - Increment total return $S(s) \leftarrow S(s) + G_t$
  - Value is estimated by mean return $V(s) = S(s)/N(s)$
- By law of large numbers,
  - $V(s) \rightarrow v_\pi(s)$ as $N(s) \rightarrow \infty$

Pseudo-code

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
    $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
    $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
    Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
            Append G to $Returns(S_t)$
            $V(S_t) \leftarrow$ average$(Returns(S_t))$

Every-Visit Monte-Carlo Policy Evaluation

- To evaluate state *s*, every time-step *t* that state *s* is visited in an episode: over a set of episodes
  - Increment counter $N(s) \leftarrow N(s) + 1$
  - Increment total return $S(s) \leftarrow S(s) + G_t$
  - Value is estimated by mean return $V(s) = S(s)/N(s)$
- Again, $V(s) \rightarrow v_\pi(s)$ as $N(s) \rightarrow \infty$
- The algorithm is the same as in previous slide except without the check for St having occurred earlier in the episode.

MC - estimation of action values

- Without a model, state values alone are not sufficient and we need to estimate the value of each action
- The policy evaluation problem for action values is to estimate $q_\pi(s, a)$
- The MC methods for this are essentially the same as just presented for state values, except now we talk about visits to a state–action pair
- A state–action pair *(s, a)* is said to be visited in an episode if ever the state *s* is visited and action *a* is taken in it
- The every-visit method estimates the value of a state–action pair as the average of the returns that have followed all the visits to it
- The first-visit MC method averages the returns following the first time in each episode that the state was visited and the action was selected

Incremental mean

The mean of a sequence can be computed incrementally:

$$
\begin{aligned}
\mu_k &= \frac{1}{k}\sum_{j=1}^{k} x_j \\
&= \frac{1}{k}\left(x_k + \sum_{j=1}^{k-1} x_j\right) \\
&= \frac{1}{k}\left(x_k + (k-1)\mu_{k-1}\right) \\
&= \mu_{k-1} + \frac{1}{k}\left(x_k - \mu_{k-1}\right)
\end{aligned}
$$

Incremental Monte Carlo updates

- Update *V(s)* incrementally **after episode** $S_1, A_1, R_2, \ldots, S_T$
- For each state S_t with return G_T

$$N(S_t) \leftarrow N(S_t) + 1$$

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}(G_t - V(S_t))$$

- Note this update is done after running the whole episode and updating “backwards”
- In non-stationary problems, it can be useful to track a running mean, i.e. forget old episodes.

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

Incremental updates for *q(s,a)*

In the case of estimation of action values the expression becomes:

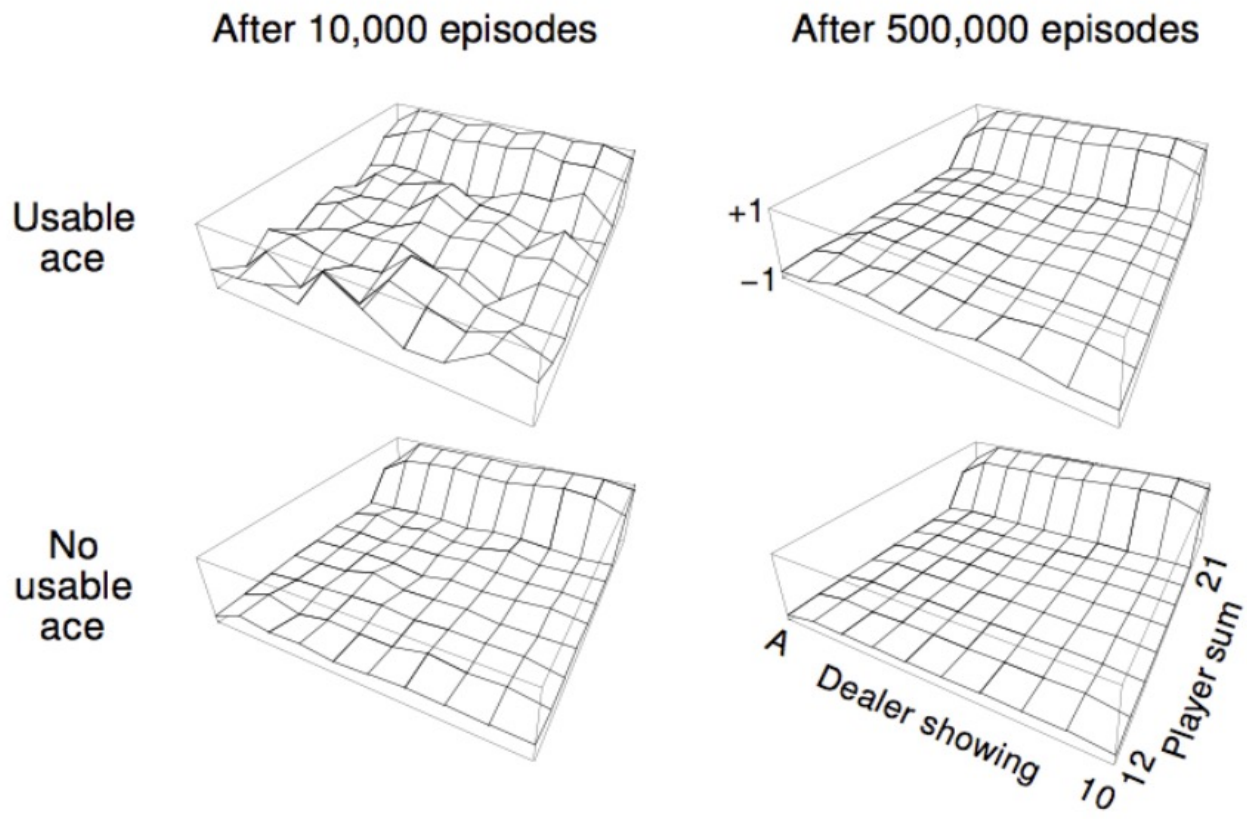
$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - Q(S_t, A_t))$$

where the counts refer to the number of visits to the pair (S_t, A_t)

Example - Blackjack

- States (200):
  - Current sum (12-21)
  - Dealer's showing card (ace-10)
  - Do I have a "useable" ace? (yes-no) – aces can have a value of 1 or 11
- Actions:
  - **stick**: Stop receiving cards (and terminate)
  - **Twist (hit)**: Take another card (no replacement)
- Rewards
  - Stick:
    - +1 if sum of cards > sum of dealer cards (WINS)
    - 0 if sum of cards = sum of dealer cards (DRAWS)
    - -1 if sum of cards < sum of dealer cards (LOSES)
  - Twist:
    - -1 if sum of cards > 21 (and terminate); 0, otherwise
- Transitions: automatically twist if sum of cards < 12

Blackjack Value Function after Monte-Carlo Learning



Policy: stick if sum of cards ≥ 20, otherwise twist

Temporal difference (TD) learning

- Temporal-difference is one of the major core ideas from RL and can be intuitively thought as combining MC and Dynamic Programming ideas !

- TD methods learn directly from episodes of experience as MC
- TD is **model-free**: no knowledge of MDP transitions / rewards as MC
- TD learns from **incomplete episodes**, by bootstrapping as DP -

TD updates its estimate (guess) based on another guess (estimate)

Monte Carlo vs Temporal Difference

- Goal: learn v_π online from experience **under policy π (**still policy evaluation)
- Incremental every-visit **Monte-Carlo**
  - Update value $V(S_t)$ toward actual return G_t

$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$$

- Simplest temporal-difference algorithm – TD(0)
  - Update value $V(S_t)$ toward estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha\,(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

  - $R_{t+1} + \gamma V(S_{t+1})$ is called the TD target
  - $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the **TD error** (difference of estimate after and before a step)

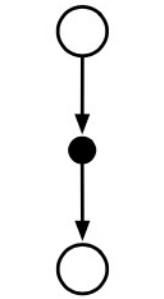
One-step TD – TD(0) algorithm

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

Pseudo-code

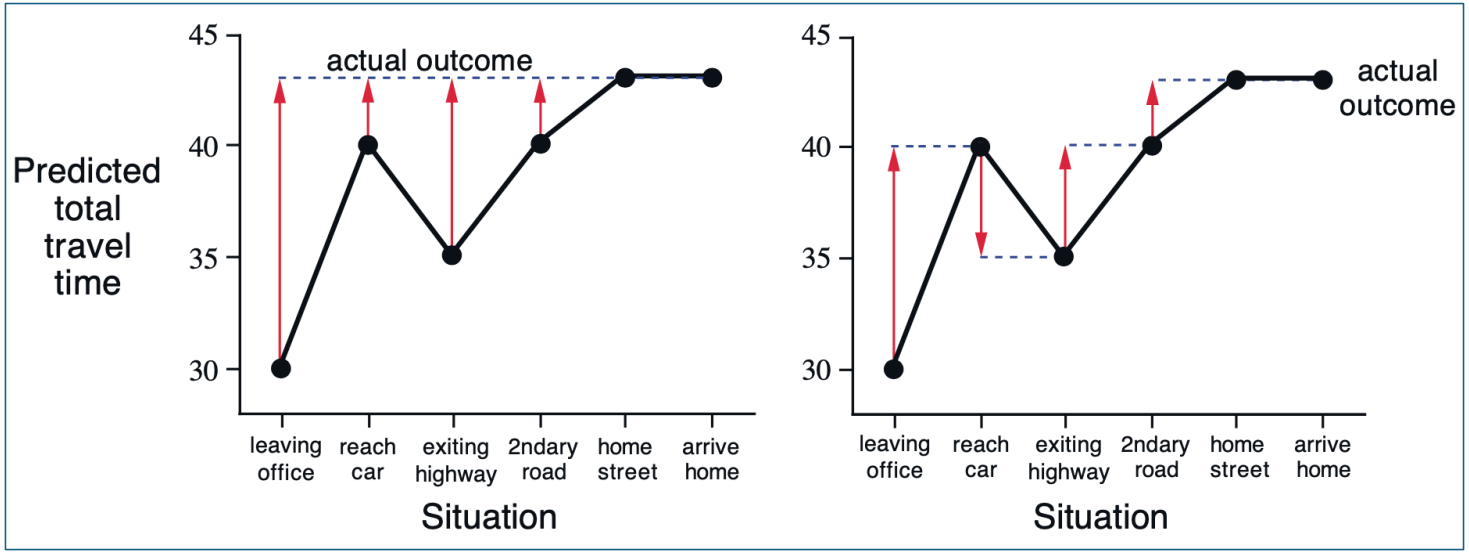


TD(0)

Example - Driving Home

State	*Elapsed Time (minutes)*	*Predicted Time to Go*	*Predicted Total Time*
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

MC vs TD



From Sutton & Barto

Advantages / disadvantages of TD/MC

- TD can learn ***before knowing*** the final outcome
  - TD can learn **online** after every step in a fully incremental way
  - MC must wait until end of episode before return is known, which may be very detrimental is many practical scenarios (e.g. long episodes)

- TD can learn ***without*** the final outcome
  - TD can learn from incomplete sequences
  - MC can only learn from complete sequences
  - TD works in continuing (non-terminating) environments
  - MC only works for episodic (terminating) environments

Bias/ variance trade-off

- Return $G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T$ is **unbiased** estimate of $v_\pi(S_t)$
- True TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ is unbiased estimate of $v_\pi(S_t)$
- But we don't know true TD, so estimated TD target $R_{t+1} + \gamma V(S_{t+1})$ is a **biased** estimate of $v_\pi(S_t)$
- TD target is **much lower variance** than the return:
  - Return depends on many random actions, transitions, rewards
  - TD target depends on one random action, transition, reward

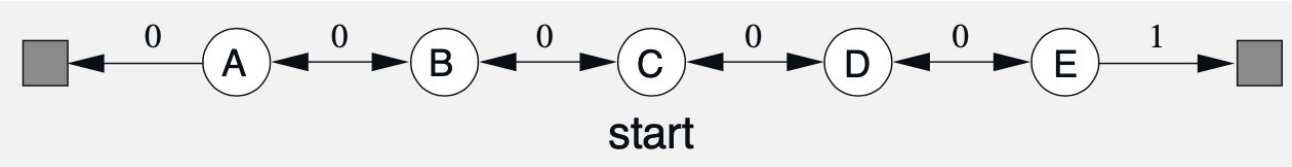
Advantages / disadvantages of TD/MC (II)

- **MC** and **TD** both converge: $V(s) \rightarrow v_\pi(s)$ as experience $\rightarrow \infty$

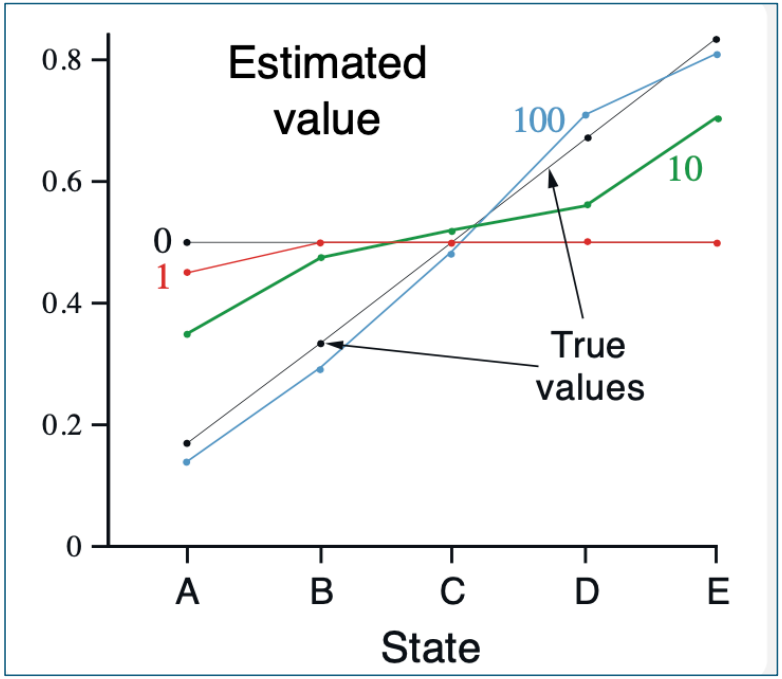
- **MC** has high variance, zero bias
  - Good convergence properties (even with function approximation for value functions – to check later)
  - Not very sensitive to initial value
  - Very simple to understand and use

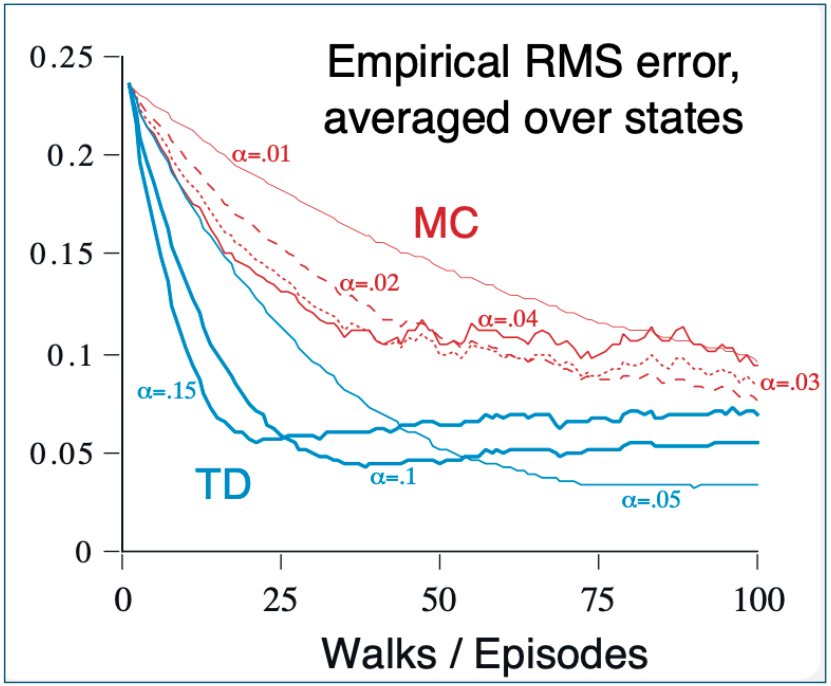
- **TD** has low variance, some bias
  - Usually, more efficient than MC
  - TD(0) **converges** to $v_\pi(s)$ (but not always with function approximation)
  - More sensitive to initial value

Example - Random walk



MC vs TD





Batch MC and TD

- Scenario where there is a fixed maximum amount of experience (e.g. k episodes); then, we can present the same episodes repeatedly – batch learning

$$s_1^1, a_1^1, r_2^1, \ldots, s_{T_1}^1$$

$$\vdots$$

$$s_1^K, a_1^K, r_2^K, \ldots, s_{T_K}^K$$

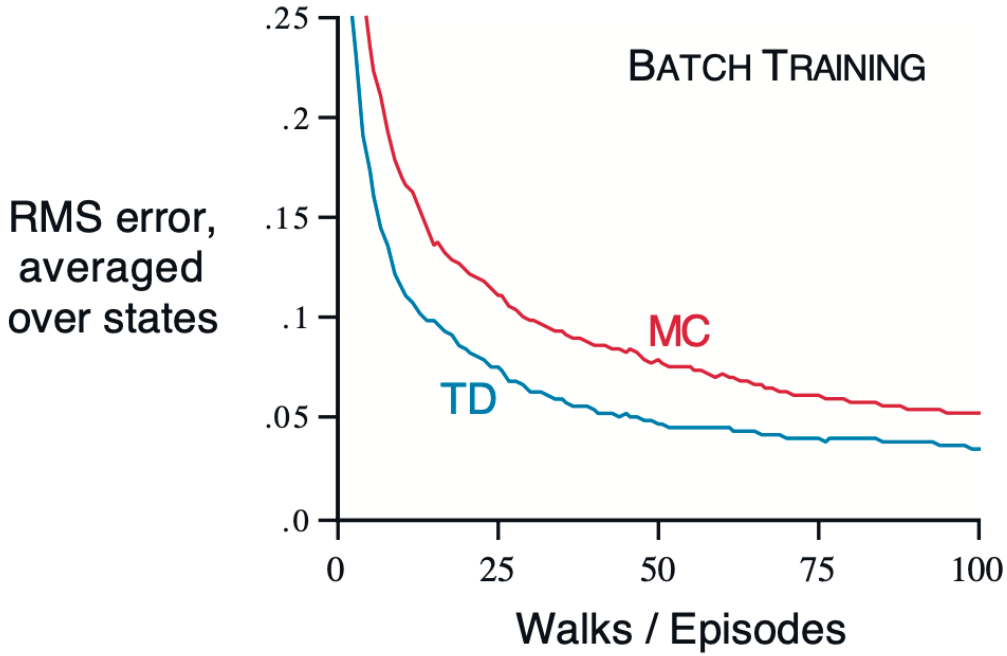
Repeating the same episodes

- Updates are done only when the whole available experience (batch) is complete
- In this case, MC and TD(0) are guaranteed to converge regardless of α, but not necessarily to the same answer
- Batch version can help to understand better the way the methods work

Random walk – batch updating

After each new episode, all episodes seen so far were treated as a batch.

RMS – difference for the true value function



Example - AB

Two states – A and B; no discount; 8 episodes

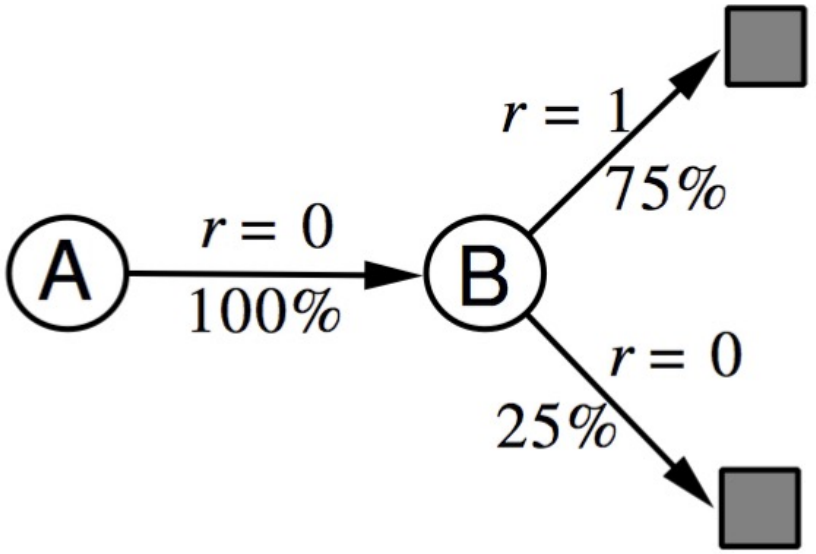
A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0

What is V(A), V(B) ?

Example - AB

Two states – A and B; no discount; 8 episodes

A, 0, B, 0
B, 1
B, 1
B, 1
B, 1
B, 1
B, 1
B, 0



What is V(A), V(B) ?

Certainty equivalence

- MC converges to solution with **minimum MSE** (on the training set)
  - Best fit to the observed returns
  - In the AB example – V(A) = 0

$$\sum_{k=1}^{K}\sum_{t=1}^{T_k}\left(G_t^k - V(s_t^k)\right)^2$$

- TD(0) converges to the solution of **Max Likelihood Markov Model**
  - Solution to the MDP that best fits the data
  - TD(0) converges to the certainy-equivalence estimate (the one estimated by the MDP)
  - In the AB example - V(A) = 0.75

$$\hat{\mathcal{P}}_{s,s'}^a = \frac{1}{N(s,a)}\sum_{k=1}^{K}\sum_{t=1}^{T_k}\mathbf{1}(s_t^k, a_t^k, s_{t+1}^k = s, a, s')$$

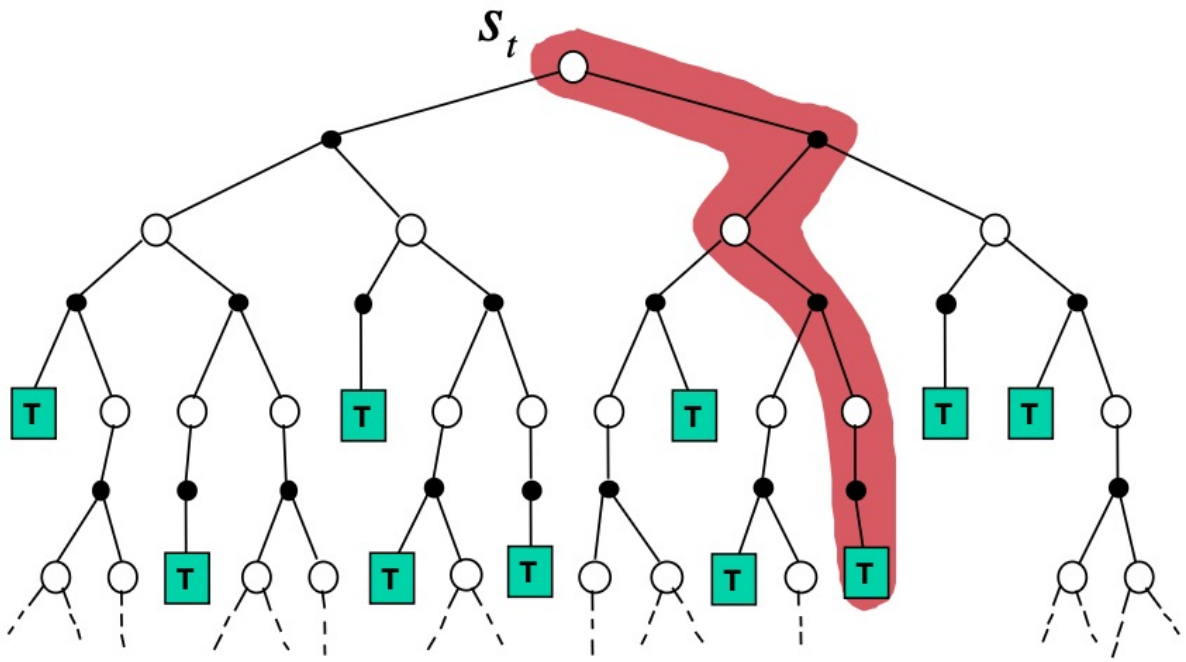
$$\hat{\mathcal{R}}_s^a = \frac{1}{N(s,a)}\sum_{k=1}^{K}\sum_{t=1}^{T_k}\mathbf{1}(s_t^k, a_t^k = s, a)r_t^k$$

Advantages / disadvantages of TD/MC (III)

- TD **exploits Markov property**
  - Usually, more efficient in Markov environments
- MC **does not exploit Markov property**
  - Usually, more effective in non-Markov environments

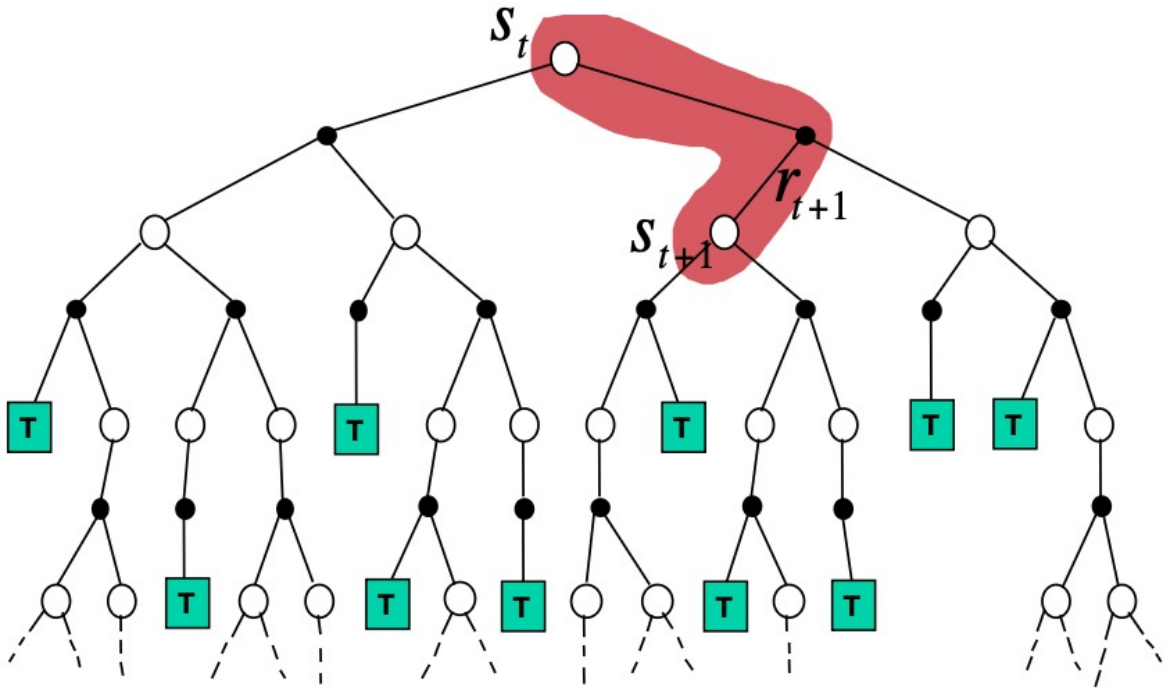
Monte-Carlo backup

$$V(S_t) \leftarrow V(S_t) + \alpha\,(G_t - V(S_t))$$



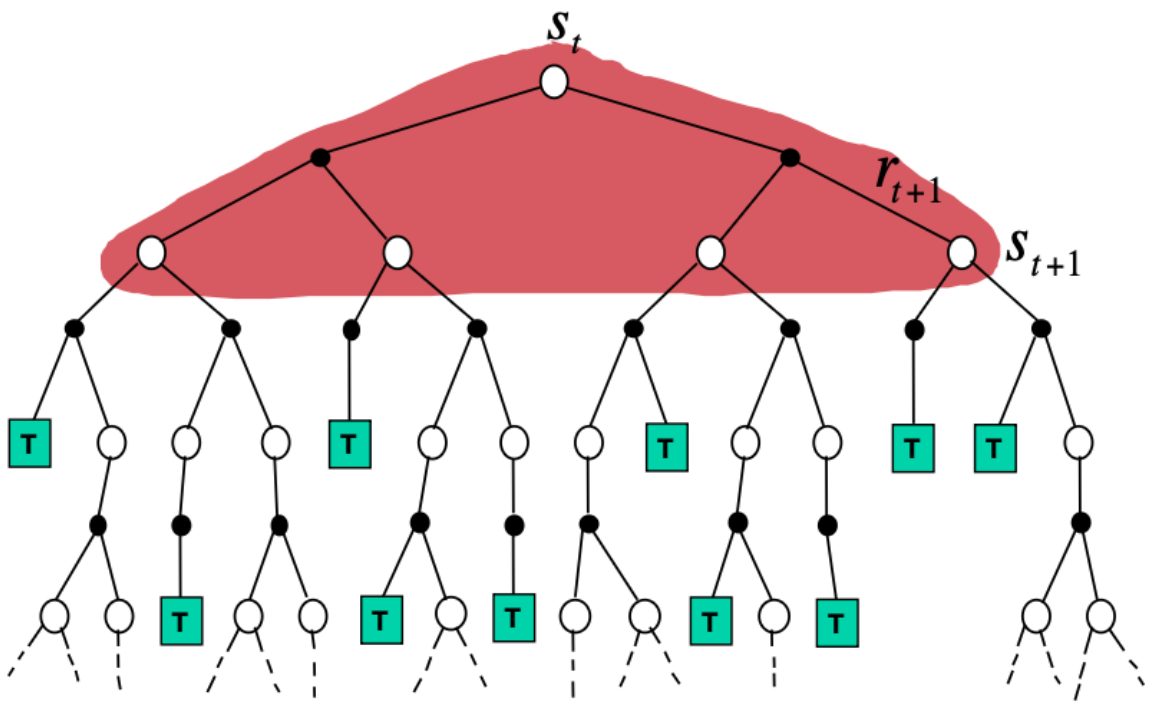
TD backup

$$V(S_t) \leftarrow V(S_t) + \alpha\left(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right)$$



Dynamic Programming backup

$$V(S_t) \leftarrow \mathbb{E}_\pi \left[R_{t+1} + \gamma V(S_{t+1}) \right]$$

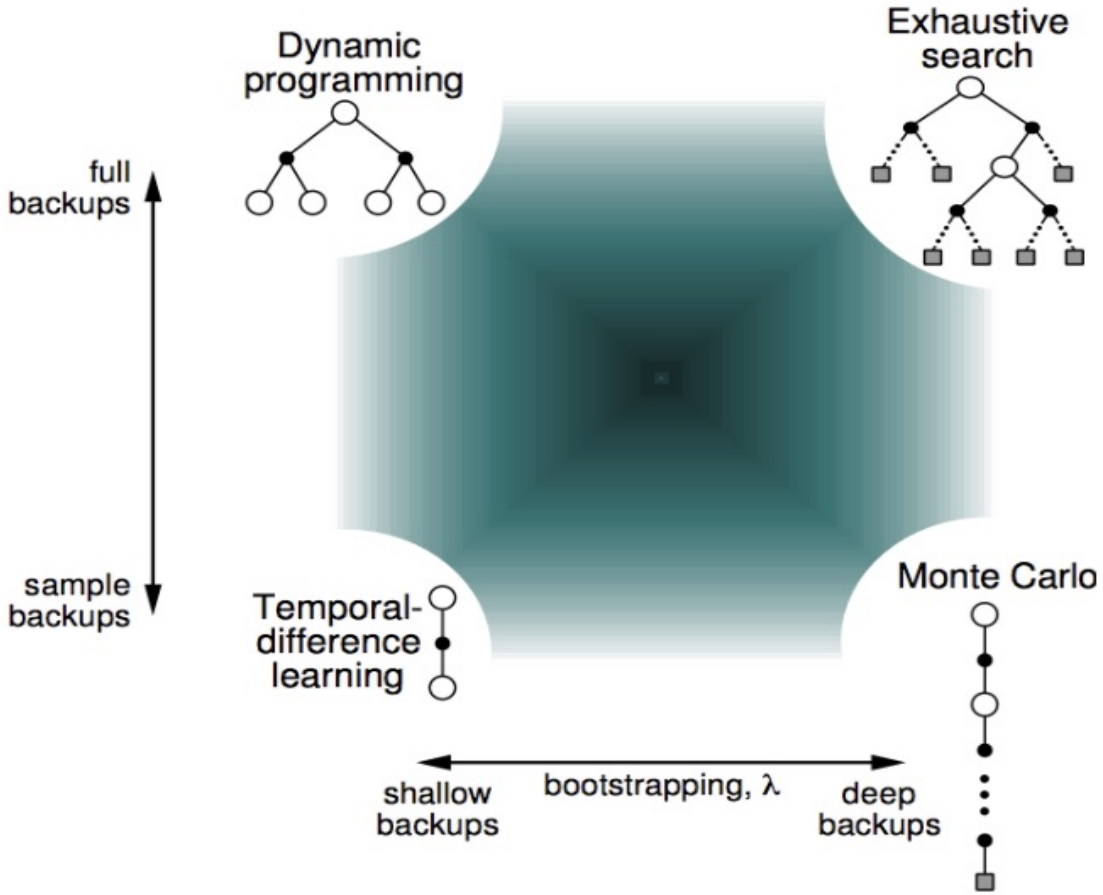


Bootstrapping and sampling

- **Bootstrapping**: update based on an estimate
  - MC does not bootstrap
  - DP bootstraps (based on recursive backup – Bellman equations)
  - TD bootstraps (based on recursive backup – Bellman equations)

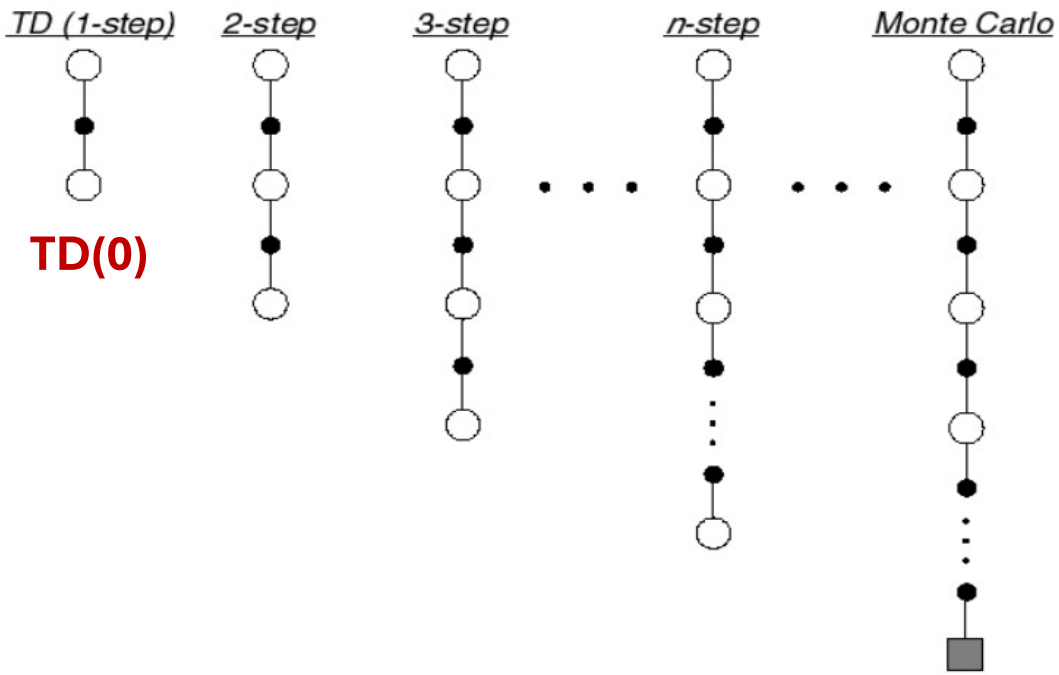
- **Sampling**: update samples an expectation
  - MC samples
  - DP does not sample (expected value calculated exhaustively for all actions from a state)
  - TD samples

Unified view of Reinforcement Learning



n-step TD prediction

Let TD target look *n* steps into the future – provides intermediate methods between TD(0) and MC



n-step return

- Consider the following n-step returns for n = 1,2,..., ∞:

$$
\begin{array}{lll}
n=1 & \textbf{TD(0)} & G_t^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \\
n=2 & & G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \\
\vdots & & \vdots \\
n=\infty & (MC) & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-1} R_T
\end{array}
$$

- Define the ***n*-step return**

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **n-step TD learning**

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^{(n)} - V(S_t) \right)$$

n-step TD prediction: algorithm

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n+1$

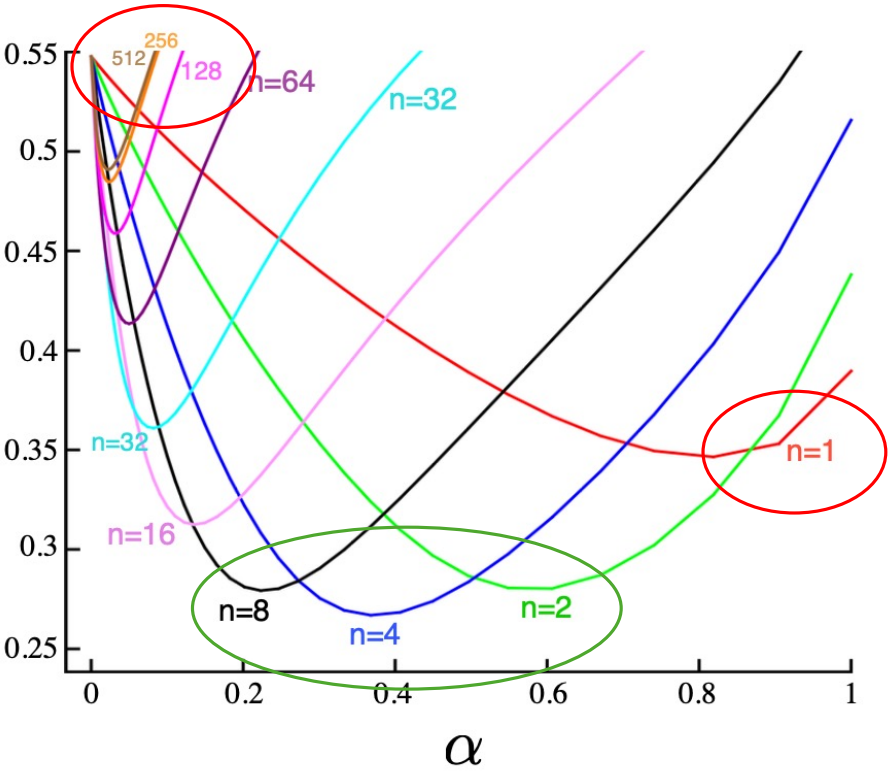
Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t+1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$
  Until $\tau = T - 1$

$G_t^{(n)}$

Pseudo-code

Example – large random walk

Average RMS error over 19 states and first 10 episodes



Intermediate values for n seem to work best

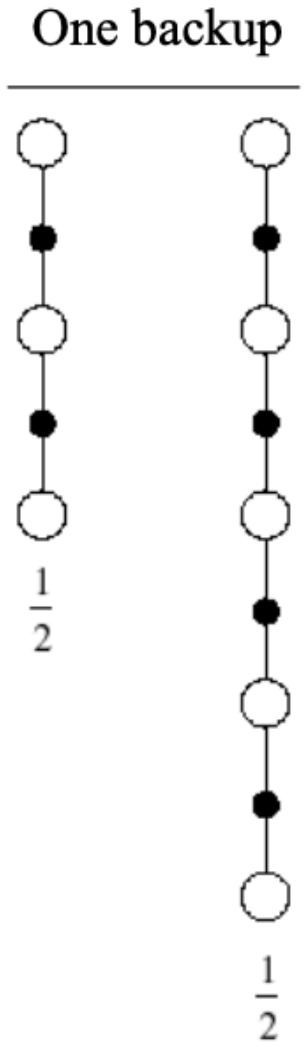
Averaging n-step returns

- We can average n-step returns over different n - e.g. average the 2-step and 4-step returns

$$\frac{1}{2}G^{(2)} + \frac{1}{2}G^{(4)}$$

- Combines information from two different time-steps

- Can we efficiently combine information from all time-steps?



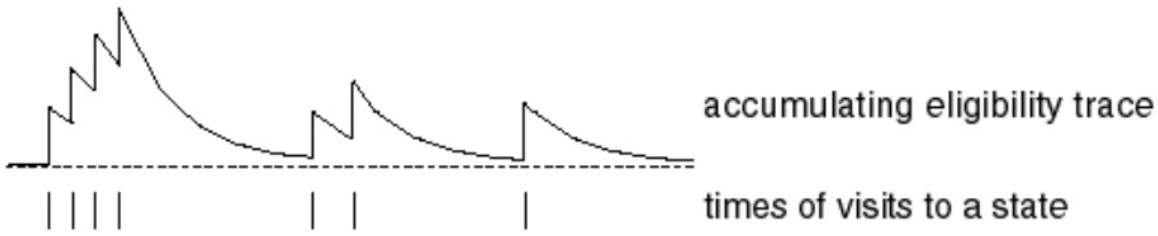
Eligibility traces

- Eligibility traces are one of the basic mechanisms of RL
- Eligibility traces unify and generalize TD and Monte Carlo methods
- TD methods augmented with eligibility traces produce a family of methods spanning a spectrum that has Monte Carlo methods at one end and TD(0) in the other, with computational advantages over n-step methods
- Eligibility traces also provide a way of implementing Monte Carlo methods online and on continuing problems without episodes.

Eligibility traces

- *Credit assignment problem: did bell or light cause shock?*
- **Frequency heuristic**: assign credit to most frequent states
- **Recency heuristic**: assign credit to most recent states
- Eligibility traces combine both heuristics

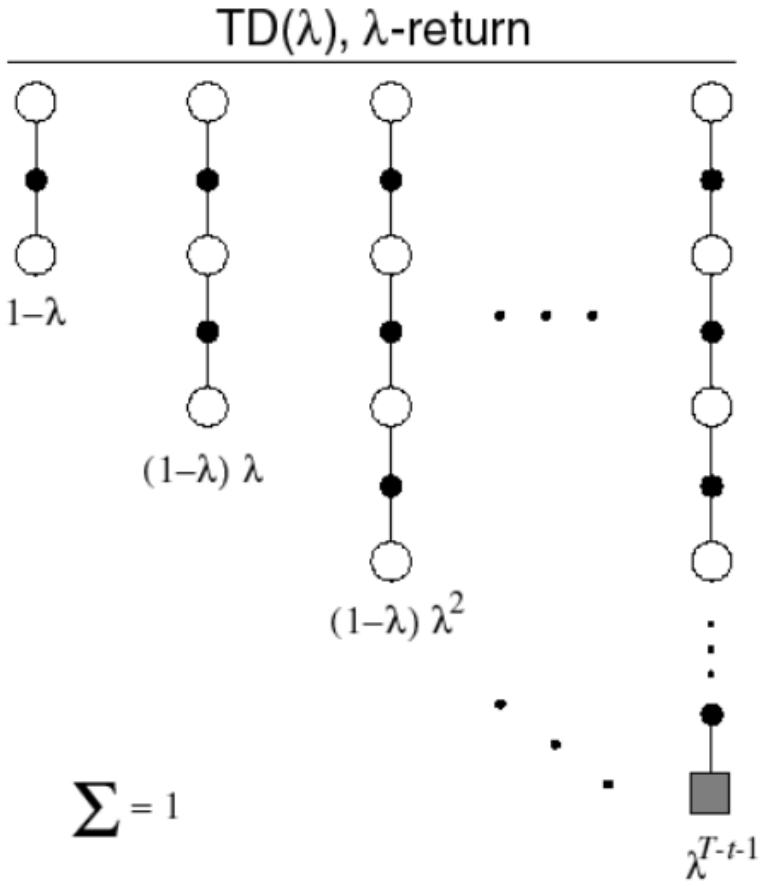
$$E_0(s) = 0$$
$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}(S_t = s)$$



E is a vector for each state

λ is the trace-decay parameter

λ-return



The λ-return G_t^λ combines all n-step returns $G_t^{(n)}$

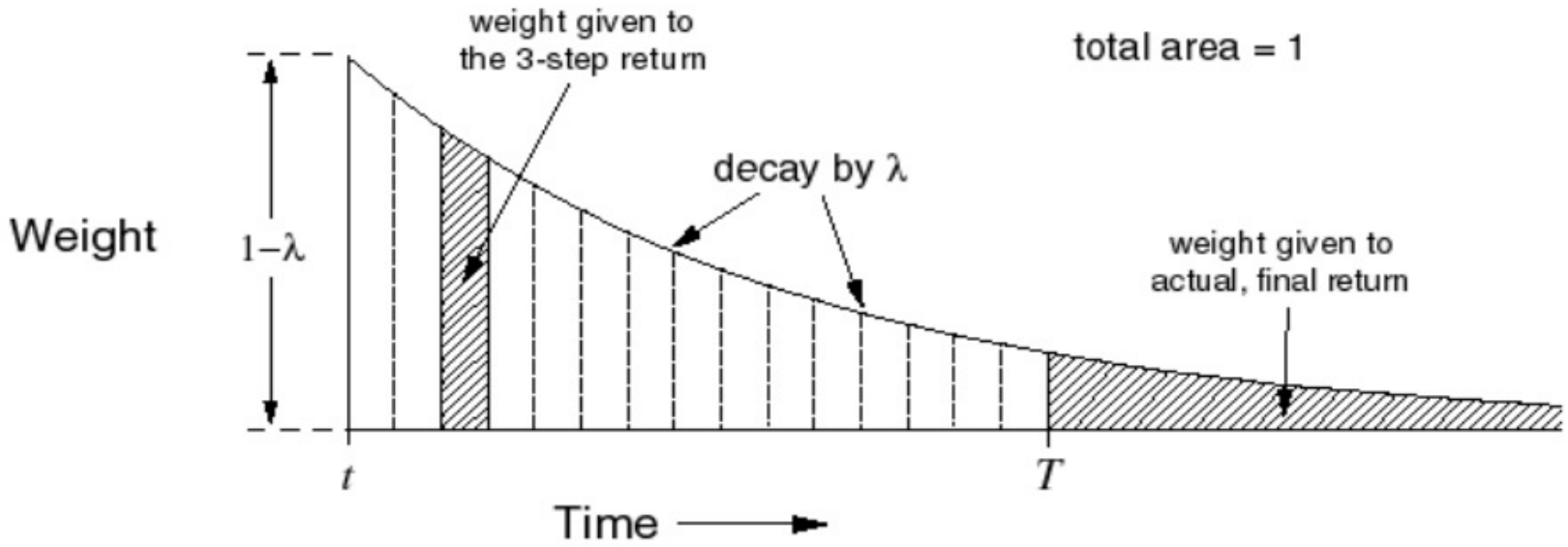
Using weight $(1-\lambda)\,\lambda^{n-1}$

$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)}$$

Forward view TD(λ)

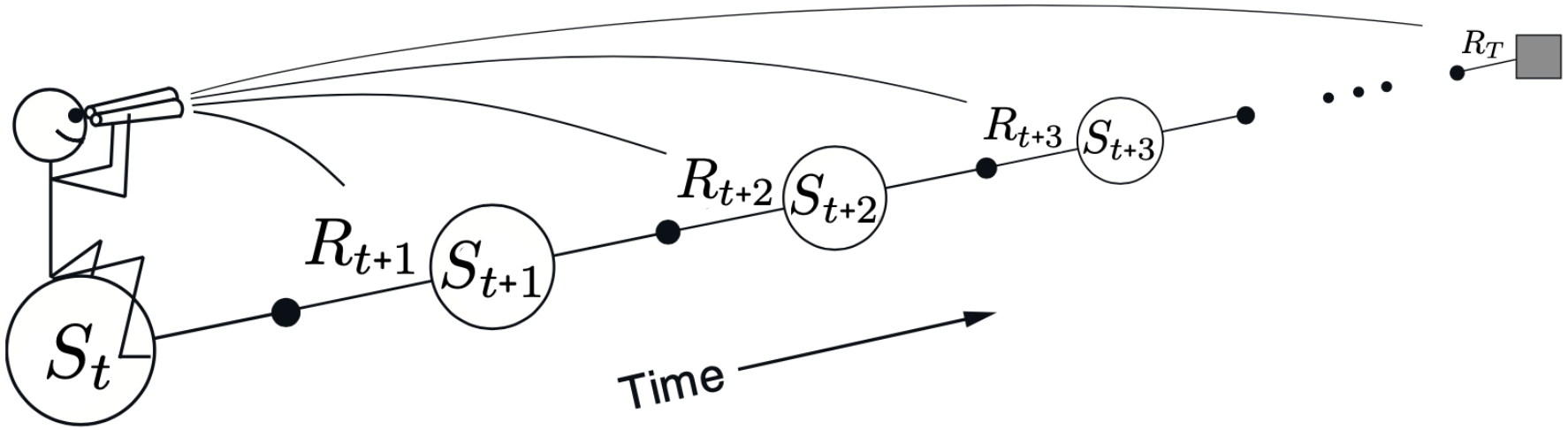
$$V(S_t) \leftarrow V(S_t) + \alpha\left(G_t^\lambda - V(S_t)\right)$$

TD(λ) weighting function



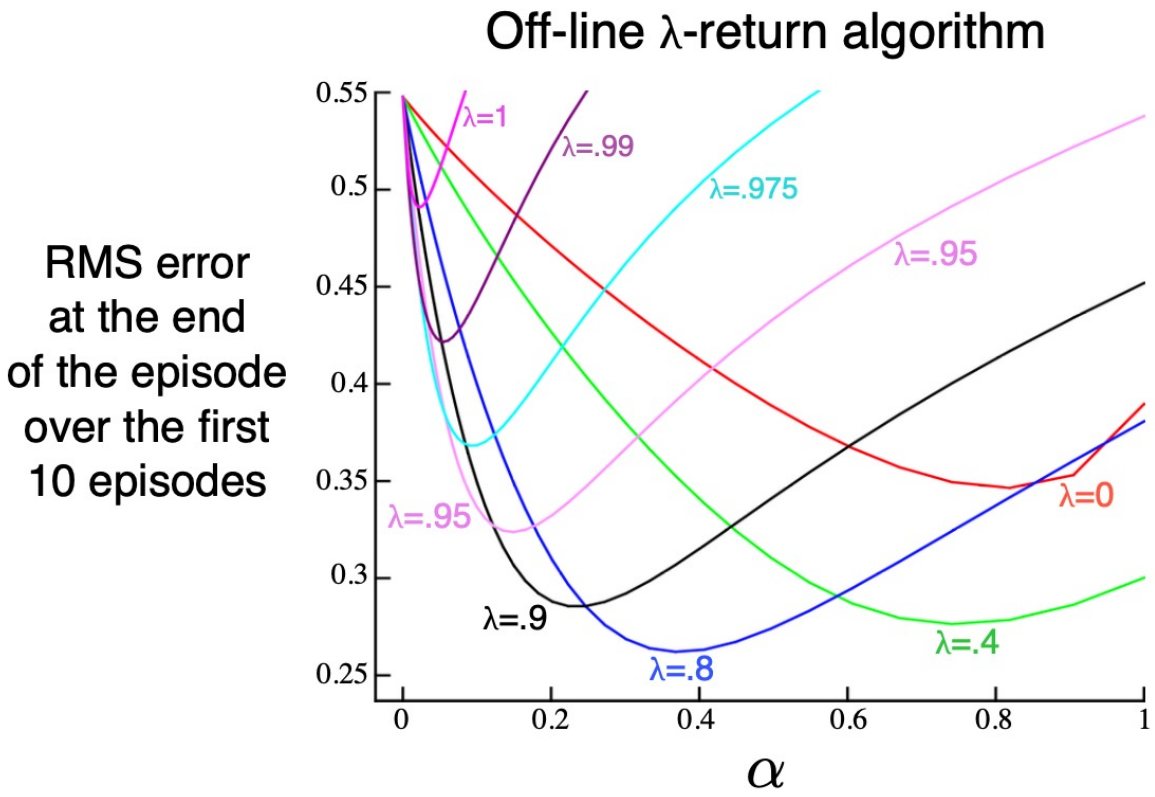
$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)}$$

Forward-view TD(λ)



- Update value function towards the λ-return
- Forward-view looks into the future to compute G_t^λ
- Like MC, **can only be computed from complete episodes**

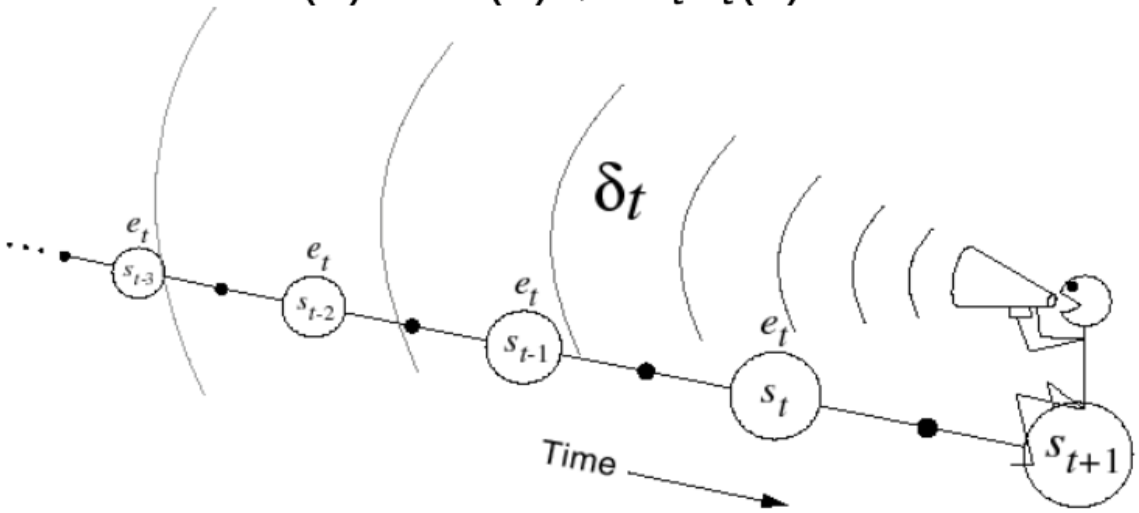
Forward-view TD(λ) on large random walk



Backward-view TD(λ)

- Keep an eligibility trace for every state *s*
- Update value *V(s)* for every state *s*
- In proportion to TD-error δ_t and eligibility trace $E_t(s)$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$
$$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s)$$



TD(λ) and TD(0)

- When λ= 0, only current state is updated

$$E_t(s) = \mathbf{1}(S_t = s)$$
$$V(s) \leftarrow V(s) + \alpha\delta_t E_t(s)$$

- This is exactly equivalent to TD(0) update

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t$$

TD(λ) and MC

- When λ=1, credit is deferred until end of episode (MC)
- Consider episodic environments with offline updates
- Over the course of an episode, total update for TD(1) is the same as total update for MC
- **Theorem**: The sum of offline updates is identical for forward-view and backward-view TD(λ)

$$\sum_{t=1}^{T} \alpha \delta_t E_t(s) = \sum_{t=1}^{T} \alpha \left(G_t^{\lambda} - V(S_t) \right) \mathbf{1}(S_t = s)$$